

AVALOKA: AN AGENTIC WORKFLOW FOR CONVERSATIONAL ETL, SCHEDULING, AND ML TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

The rise of generative AI has paved the way for systems that automate many of the deterministic and repetitive tasks involved in ETL, analytics, and machine learning workflows. However, such systems must be built while preserving the human way of oversight and intent. Although large language models have demonstrated capabilities in assisting with these workflows, achieving consistently high-quality automation demands careful engineering, structured constraints, and a deliberate separation of responsibilities. Avaloka, a multi-agentic orchestration platform, addresses this need by coordinating specialized agents that collaborate to automate mundane and execution-heavy tasks, while enabling users to remain active participants through a conversational interface. Non-deterministic tasks that require human judgment—such as planning and validation—are handled through LLM-assisted reasoning guided by rigorously designed constraints, machine learning principles that emulate human reasoning, and high-quality prompting, ensuring reliability rather than ad-hoc automation.

Avaloka’s agent collaboration is orchestrated using LangGraph, enabling structured and inspectable workflows in which each agent is designed with a well-defined responsibility. Reasoning-focused agents synthesize plans and validations, while execution-focused components are delegated their operations, ensuring a clear separation between decision-making and action. Built for distributed processing, scalable infrastructure provisioning, and flexible scheduling, this platform allows analytics and ML workloads to handle enterprise-grade data volumes and implementation requirements. Avaloka provides a platform that transforms the raw capabilities of generative AI into a dependable, assistive system for highly impactful analytics and machine learning solutions.

1 INTRODUCTION

Natural language interfaces are making it easier for more people to work with data. By letting users describe what they want in plain English, these tools help bridge the gap between ideas and real data workflows. They simplify tasks like ETL, analytics, and machine learning. But turning a conversation into a finished, reliable data pipeline is not simple. Large language models (LLMs) can give different answers each time, data processes need careful checks before they run at scale, and companies need strong controls for things like scheduling, setting up infrastructure, and tracking what gets created.

Many current tools that use LLMs for data workflows mix up the planning stage with actually running the code. This can lead to fragile systems—sometimes untested code is run directly on production data, or complicated tasks do not have easy ways for people to check each step. Plus, a lot of conversational AI systems try to do everything at once, without giving users a chance to review, refine, or validate the process. Real-world data work usually involves back-and-forth improvements, repeated checks, and tools for scheduling and tracking, which these systems often miss.

Avaloka addresses these limitations through a principled multi-agent architecture that separates reasoning from execution and enforces validation gates at each transition by:

1. **Staged validation with sample-based reasoning.** Before any code touches production data, it undergoes syntax compilation, static semantic analysis, logic review, and execution

samples on two rows. This ensures that expensive compute resources are allocated only after validation succeeds.

2. **Graph orchestration for explicit state transitions.** Avaloka uses LangGraph to implement workflows as directed acyclic graphs, with nodes representing agents and edges encoding conditional routing logic. Because of this structure, users can view the intended workflow as a Graphviz diagram and examine and debug execution traces.
3. **Enterprise deployment through operational integration.** Avaloka facilitates multi-database sampling (MCP servers), distributed execution, tracking ML experiments, and periodic task scheduling in addition to one-time conversational queries. Without requiring users to switch between tools in different contexts, these integrations guarantee that the system scales from exploratory analysis to production workflows.

The remainder of this paper is organized as follows: Section 2 provides a system overview and architectural rationale; Section 3 details the agentic workflow, including agent responsibilities, validation tiers, multi-dataset handling, and routing logic; Section 4 describes scheduling and long-running task management; Section 5 covers the model training agent and inference pipeline; Section 6 presents the data transfer agent for large-scale migrations; Section 7 presents implementation details and the technology stack; Section 8 evaluates the system through usage scenarios and test coverage; Section 9 addresses safety, reliability, and limitations; and Section 11 concludes with future directions.

What sets Avaloka’s approach apart from existing approaches is that, unlike single systems that generate code in one shot (e.g., Copilot), Avaloka enforces multi-tier validation and iterative refinement with explicit feedback loops. Unlike workflow automation platforms like Airflow that require manual DAG authoring, Avaloka architects workflows from natural language while retaining human monitoring. Avaloka integrates operational tooling and scales from sampling to production execution without manual intervention, in contrast to notebook-based LLM assistants. Avaloka stands out as a production-grade agentic system thanks to its combination of conversational intent capture, thorough validation, and operational readiness.

2 SYSTEM OVERVIEW

Avaloka is engineered as a distributed, multi-agentic orchestration platform designed to bridge the gap between high-level human intent and low-level data execution. The system’s architecture is rooted in the principle of decoupling reasoning from execution, ensuring that non-deterministic LLM outputs are rigorously validated before interacting with enterprise-grade data.

Avaloka’s agent ecosystem has two layers to ensure high quality execution workflows:

Decision-Making (Reasoning) Layer. This layer comprises agents (such as Planner, Summariser, Coder, Validator, etc.) which synthesize strategies, draft code, and perform cross-validations (logical and execution-based) on sampled data subsets. No large-scale data mutation occurs at this stage.

Execution Layer. Once a plan is validated by the system (and the user, if involved), the system transitions to execution-focused components. Agents for model training, large-scale execution of analytics, and data transfer take over, following the pipelines established by the reasoning layer. This ensures that expensive compute resources are only utilized once the operational logic is verified.

The core of Avaloka is a LangGraph-based runtime that manages the lifecycle of these agents.

Graph-based transitions. Each agent is represented as a node in a directed graph, where edges represent explicit state transitions. This structure allows for complex branching logic, such as looping back to the Coder agent if the Validator detects a syntax or logical error.

Shared state management. Communication between agents is facilitated through a robust shared state (state schema). Instead of passing massive message histories, agents update a central, versioned state object. This allows specialized agents to access only the context they need, maintaining efficiency even in complex workflows.

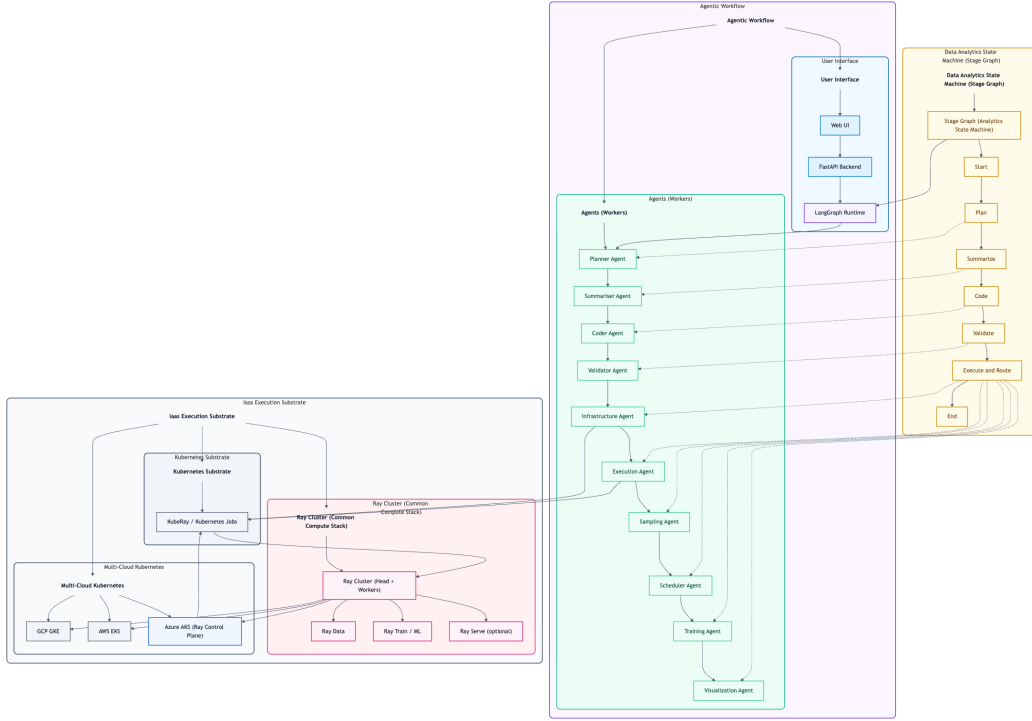


Figure 1: Avaloka architecture diagram showing the agentic workflow, state machine, and execution substrate.

Model Context Protocol (MCP) server. Avaloka incorporates an MCP server to *[placeholder: describe MCP server role]*.

The reasoning layer is responsible for translating user objectives into a verifiable technical strategy. This layer operates by utilizing data samples, rather than full production sets, to ensure safety and cost-efficiency.

Planner agent. The Planner agent serves as the primary architect of the mission. It parses the natural language request to identify the core objective, determines the specific sequence of agents required, and defines the coordination logic between them.

Coder agent. The Coder agent synthesizes the code by emulating the human way of thinking and coding. It drafts pseudo-code based on the Planner’s instructions and the user’s original prompt while inspecting a representative preview sample of the dataset. Using the pseudo-code as a formal guide, the agent then generates the final executable code.

Validator agent. The Validator agent ensures the generated code is both safe and correct through a tiered validation process, where the code is checked for logical consistency and syntax errors to prevent runtime failures.

Execution agent. The Execution agent performs a dry run on a small data sample. This allows the system to verify the output format and catch execution-level bugs without incurring the cost of a full-scale run.

To handle the execution-heavy tasks of ETL and ML, Avaloka integrates specialized high-performance libraries with its infrastructure provisioning in the following way:

Daft (lazy dataframe processing). We utilize Daft for its specialized ability to handle larger-than-memory datasets. By employing a lazy execution engine, Daft allows Avaloka to define com-

plex data transformations without immediate materialization, which is critical for processing the petabyte-scale data volumes common in industrial environments (7).

Ray (distributed execution). To ensure horizontal scalability, Ray serves as the backbone for distributed parallelization. It allows Avaloka to distribute training and ETL workloads across a cluster of nodes, ensuring that agent-orchestrated tasks are not bottlenecked by single-node compute limits (6).

Infrastructure agent. This component is responsible for the dynamic provisioning of cloud resources (e.g., GKE or EKS clusters) based on user constraints such as budget and performance requirements. The Infrastructure agent manages the scaling of these resources to support large-scale runs (10GB+ scale). It is capable of executing Ray workers in parallel on Kubernetes.

Scheduling. For periodic or deferred workloads, Avaloka utilizes Celery and RedBeat to *[placeholder: describe scheduling behavior]*.

The actual processing of enterprise-scale data is managed by specialized execution agents. These agents bridge the gap between the validated Python logic and the distributed compute infrastructure (Ray/Kubernetes).

Data Transfer Agent (DTA). The DTA is responsible for the movement of large volume datasets across heterogeneous environments. It abstracts the complexity of connecting to data sources and the ETL transformation and movement logic by using the same underlying Planner-Coder-Validator loop to plan the workflow. For ETL workflows, the DTA ensures data integrity during migration and handles the parallelized streaming of 10GB+ datasets to prevent memory bottlenecks.

Model Training Agent (MTA). The MTA manages the end-to-end lifecycle of machine learning workloads. *[placeholder: describe training agent responsibilities]*.

Avaloka preserves human oversight through two primary mechanisms:

Refinable DAGs. During the planning phase, the system translates user intent into an executable plan, which can be rendered as a directed acyclic graph (DAG) called the Planner Graph, which users can inspect, modify, or reject before execution begins.

Visualization agent. A dedicated agent leverages Apache Charts to *[placeholder: describe visualization role]* and provide visual insights during both the data sampling phase and final result synthesis.

Given the long-running nature of ML training and enterprise ETL jobs, Avaloka ensures system resilience by *[placeholder: describe checkpointing or recovery behavior]*. This allows the system to resume from the last successful node in the LangGraph should a hardware or network failure occur, preventing costly re-computation.

3 AGENTIC WORKFLOW

The agent roster includes:

- Planner: collects requirements and chooses the next action.
- Planner Graph: produces a mermaid-compatible DAG for inspection.
- Summariser: converts the conversational plan into a structured job contract.
- Coder: generates pseudo-code and then executable Python.
- Validator: performs syntax, static-semantic, and logical checks.
- Execution: runs code locally or on Kubernetes and captures artifacts.
- Visualization: creates charts from execution outputs.
- Scheduler: manages periodic runs and task operations.

- Model Training Agent (MTA): builds training plans and runs training/inference.

These agents exchange a shared state object, ensuring that validation results, artifacts, and task metadata are propagated across steps.

4 SCHEDULING AND LONG-RUNNING TASKS

Avaloka includes a Scheduler agent built on Celery and RedBeat. Users can request periodic jobs using cron-style parameters, retrieve task status, fetch prior results, or cancel scheduled tasks. Scheduled runs reuse the existing pipeline by passing serialized state into the execution path. This design keeps scheduled jobs consistent with the conversational flow and its validation gates.

5 MODEL TRAINING AND INFERENCE

The Model Training Agent provides autonomous training workflows, integrating MLflow for experiment tracking and model artifact management (?). Training can be initiated directly or scheduled for later execution. The routing logic prioritizes inference requests, then training plan requests, and finally code execution. This ordering prevents training intent from being misrouted to code generation.

6 DATA TRANSFER AGENT

The Data Transfer Agent (DTA) represents an execution specialization within Avaloka, designed to resolve one of the most persistent mundane bottlenecks in enterprise data engineering: the manual movement and transformation of large-scale data.

The motivation for the DTA stems from the observation that significant time is consumed by data plumbing: tasks such as migrating data between storage buckets, re-partitioning datasets, or performing routine cleaning, format conversions, and columnar transformations. While these tasks are deterministic, they are high-stakes, as a single manual coding error in a migration script can lead to widespread issues at scale.

The DTA addresses this by treating data transfer as an agentic task that adheres to the same rigorous contract as Avaloka’s reasoning layer. By leveraging AI to automate the repetitive aspects of data management, the system renders data lakes and warehouses more amenable to downstream analytics and machine learning, eliminating the need for manual human intervention for every transfer.

A core architectural principle of Avaloka is that the execution engine remains swappable without altering high-level agent behavior. To maintain this, the DTA implements a dedicated pipeline that mirrors the core Avaloka loop—Plan, Code, Validate, and Execute—before committing to a full-scale transfer.

To handle the demands of enterprise-grade volumes, the DTA utilizes a high-performance stack of Daft which employs lazy execution to accommodate larger-than-memory transformations, and Ray and Kubernetes for distributed and scalable transfers (7; 6).

7 IMPLEMENTATION DETAILS

Avaloka is implemented in Python with a Streamlit UI and a FastAPI backend. Key dependencies include LangGraph for orchestration, Celery/Redis for scheduling, and MLflow for model tracking. Infrastructure provisioning is supported for Kubernetes targets on GKE or EKS. The system maintains deterministic fallbacks so workflows continue to completion even when external model endpoints are unavailable. For LLM-backed planning, coding, summarization, validation, and visualization, the current configuration uses Groq-hosted Llama models (Llama 3.3 70B Versatile and Llama 3.1 8B Instant) (4; 5).

7.1 RAY AND DAFT IN AVALOKA

Apache Ray provides a distributed execution substrate with task scheduling, actor-based stateful services, and elastic scaling for Python workloads (6). This fits Avaloka’s need to scale agent execution and long-running pipelines beyond a single host, while preserving a unified control plane for scheduling, retries, and resource-aware execution. In future iterations, Ray can serve as the execution backend for heavy ETL steps and model training tasks, allowing Avaloka to route compute-intensive stages to Ray clusters while keeping planning and validation local.

Daft is a Python dataframe library designed for high-performance, lazy, and distributed data processing, with integrations for cloud object storage and columnar formats (7). Its dataframe-first API aligns with Avaloka’s code generation patterns and validation steps, enabling generated transformations to map cleanly to a scalable execution engine. In practice, Daft can support larger-than-memory datasets and heterogeneous storage backends, making it a natural fit for Avaloka’s ETL workloads that progress from sampling to full-scale execution.

8 EVALUATION AND USAGE SCENARIOS

We provide a comprehensive test suite covering the scheduler, execution pipeline, and training workflows. Qualitatively, Avaloka supports:

- Conversational ETL planning with validation-backed code generation.
- Periodic data refresh workflows with automatic status and result retrieval.
- Model training requests with tracking and artifact outputs.
- End-to-end runs that culminate in visualization for quick insight.

A quantitative benchmark and user study are left for future work. Candidate benchmarks include DS-STAR and DABstep (2; 3).

9 SAFETY AND RELIABILITY

Reliability is enforced via layered validation, deterministic fallbacks, and explicit execution routing. Task scheduling is isolated through Celery workers and Redis-backed metadata. The system also avoids unintended execution by requiring explicit signals before transitioning between planning, coding, training, and execution phases.

10 LIMITATIONS

Avaloka currently assumes access to configured infrastructure (Redis, Kubernetes, MLflow) for its advanced features. Task scheduling depends on available Celery workers, and long-running workflows may require additional observability tooling for production deployments.

11 CONCLUSION

Avaloka demonstrates how a multi-agent, stateful workflow can operationalize conversational data tasks while retaining validation guarantees and operational controls. The system unifies planning, coding, training, scheduling, and visualization into a single orchestration layer suitable for ETL and ML workflows.

REFERENCES

- [1] Dawn Song, Chenguang Wang, Nicholas Crispino, Ruoxi Jia, Kyle Montgomery, Yujin Potter, Vincent Siu, and Zhun Wang. *Agents in the Wild: Safety, Security, and Beyond*. arXiv:2508.05002, 2025. .

- [2] Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Tomas Pfister. *DS-STAR: Data Science Agent via Iterative Planning and Verification*. arXiv:2509.21825, 2025. .
- [3] Alex Egg et al. *DABstep: Data Agent Benchmark for Multi-step Reasoning*. arXiv:2506.23719, 2025. .
- [4] GroqDocs. *Llama 3.3 70B Versatile (model identifier:)*. .
- [5] GroqDocs. *Llama 3.1 8B Instant (model identifier:)*. .
- [6] Apache Ray documentation. .
- [7] Daft documentation. .